

Московский Авиационный Институт  
(Национальный Исследовательский Университет)

Факультет информационных технологий и прикладной математики  
Кафедра вычислительной математики и программирования

**Лабораторная работа №9 по курсу  
«Дискретный анализ»**

**Графы**

Студент: Эсмедляев Федор Романович

Группа: М8О–312Б–22

Преподаватель: Н.Д.Глушин

Оценка: \_\_\_\_\_

Дата: \_\_\_\_\_

Подпись: \_\_\_\_\_

Москва, 2024.

## Вариант 8

# Ф. 8 Поиск максимального паросочетания алгоритмом Куна

|                     |                                  |
|---------------------|----------------------------------|
| Ограничение времени | 10 секунд                        |
| Ограничение памяти  | 1Gb                              |
| Ввод                | стандартный ввод или input.txt   |
| Вывод               | стандартный вывод или output.txt |

Задан неориентированный двудольный граф, состоящий из  $n$  вершин и  $m$  ребер. Вершины пронумерованы целыми числами от 1 до  $n$ . Необходимо найти максимальное паросочетание в графе алгоритмом Куна. Для обеспечения однозначности ответа списки смежности графа следует предварительно отсортировать. Граф не содержит петель и кратных ребер.

## Формат ввода

В первой строке заданы  $1 \leq n \leq 110000$  и  $1 \leq m \leq 40000$ . В следующих  $m$  строках записаны ребра. Каждая строка содержит пару чисел – номера вершин, соединенных ребром.

## Формат вывода

В первой строке следует вывести число ребер в найденном паросочетании. В следующих строках нужно вывести сами ребра, по одному в строке. Каждое ребро представляется парой чисел – номерами соответствующих вершин. Строки должны быть отсортированы по минимальному номеру вершины на ребре. Пары чисел в одной строке также должны быть отсортированы.

## Пример

Ввод 

```
4 3
1 2
2 3
3 4
```

Вывод 

```
2
1 2
3 4
```

## Метод решения:

Алгоритм Куна заключается в нахождении увеличивающей цепи. Если мы найдем все увеличивающие цепи и их больше нет, означает, что мы нашли максимальное паросочетание по т.Бёржа.

Давайте просто проходимся по графу, если наша вершина имеет связь идем по этой связи, если происходит ситуация, где появляется увеличивающая цепь, то мы ее используем.

Сложность:  $O(N \cdot M)$

## Описание программы

### □ Инициализация и ввод данных:

- Считывание размеров графа:
  - Программа принимает два числа  $n$  (количество вершин) и  $m$  (количество рёбер).
  - Создаётся граф  $g$  в виде списка смежности, где каждая вершина  $u$  соответствует вектору соседей.
- Ввод рёбер графа:
  - Программа считывает  $m$  пар  $(u, v)$ , которые задают рёбра графа.
  - Для каждого ребра вершины  $u$  и  $v$  добавляются в списки смежности друг друга (так как граф неориентированный).

---

### Сортировка списка смежности:

- Для каждой вершины (строки в списке смежности) соседние вершины сортируются в порядке возрастания:
  - Это делается для упорядоченного обхода в дальнейшем.
  - Сложность сортировки:  $O(m \cdot \log(m/n))$ .

---

### Поиск максимального паросочетания:

1. Инициализация массива для паросочетания:
  - Массив `matching` размером  $n$ , где изначально все значения равны  $-1$ , чтобы указать, что вершины не соединены.
2. Цикл по вершинам:
  - Для каждой вершины  $i$ , вызывается функция DFS для поиска увеличивающего пути:
    - Цель DFS: Найти путь, который позволяет улучшить текущее паросочетание, связывая новую пару вершин.
3. Алгоритм DFS:
  - Если вершина  $u$  ещё не посещена, то она помечается как посещённая.
  - Для каждой соседней вершины  $v$ :
    - Если  $v$  ещё не связана (значение в `matching[v] == -1`) или если можно пересвязать  $v$  через увеличивающий путь, найденный в глубине:
      - Связываем  $u$  и  $v$  в массиве `matching`.

---

### Сбор результата:

### 1. Формирование пар:

- Для каждой вершины  $v$ , которая имеет соответствие в массиве `matching`, создаётся пара  $(u,v)$ , где  $u=\text{matching}[v]$ , если  $u < v$  (чтобы избежать дублирования).

### 2. Сортировка пар:

- Все пары  $(u,v)$  сортируются в порядке возрастания.

---

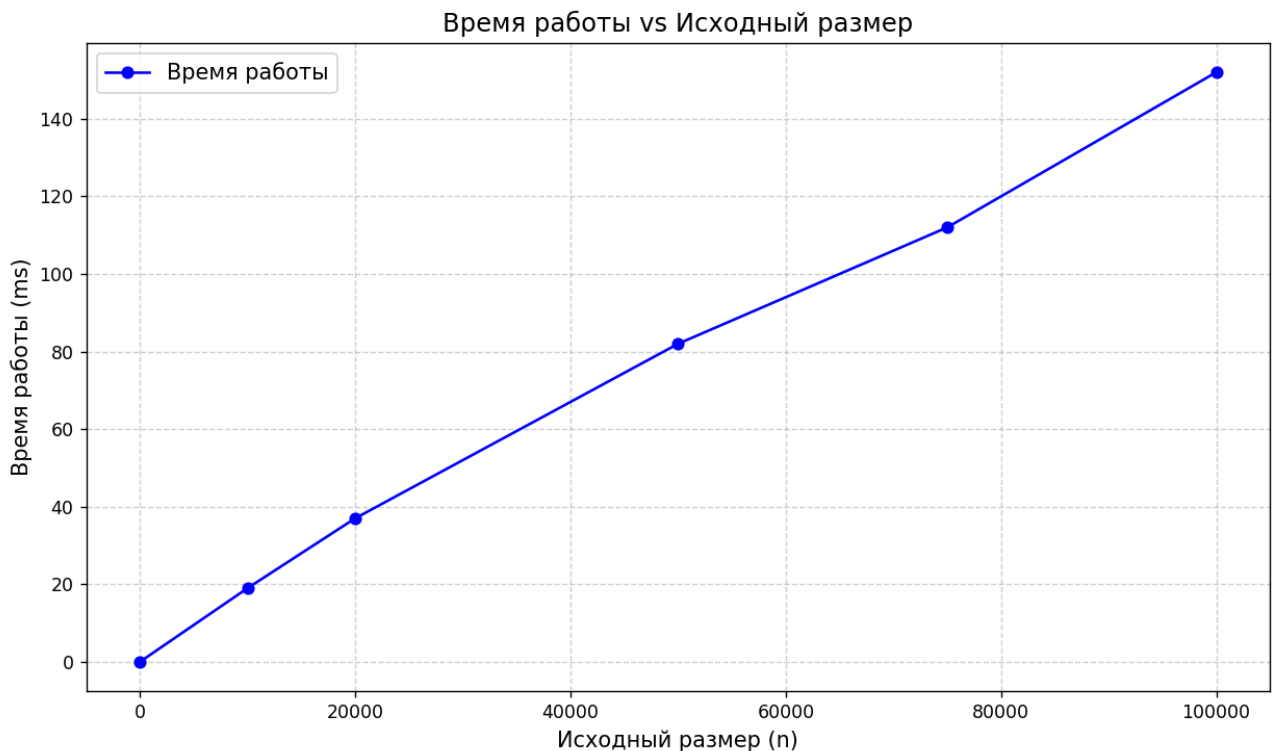
Вывод результата:

- Программа выводит:
  1. Количество пар в паросочетании.
  2. Сами пары в лексикографическом порядке.

## Дневник отладки

- 1) Проблем не было с первой отправки результат - ОК

## Тест производительности



## Вывод:

Я научился находить максимальное количество паросочетаний с помощью алгоритма Куна.